BDE PartnerBot

Edouard Souied

Autonomous B2B Partnership Outreach, Human-in-the-Loop AI Copywriting & AI-Powered Inbox Triage

Target Level: Level 3 (Advanced)	**Core Stack:** n8n, PostgreSQL, Groq (GPT-OSS 120B & 20B), Gmail (Google Workspace), Telegram	**Runtime:** Docker (local) + Cloudflare Tunnel (Windows service)

01 · The Need

The need in one sentence: the partnership team of the ESSEC BDE student union loses hours every week because it writes every sponsor email by hand in Gmail, tracks follow-ups in a Google Sheet and reads a shared inbox full of spam to find the few real partner replies.

Student union (BDE) partnership executives waste hours manually drafting personalized outreach emails, tracking prospect follow-ups across scattered spreadsheets, and sorting a shared inbox where real partner messages are buried under newsletters and spam. The result: duplicate emails, forgotten follow-ups and missed sponsorship opportunities.

What we do today instead: Copy-paste pitch templates into Gmail, maintain status tabs in Google Sheets, rely on ad-hoc reminders and read every incoming email by hand. Members frequently overwrite each other's prospects or fail to follow up within business timelines.

What the system changes: A Level 3 pipeline backed by PostgreSQL and two Groq-hosted LLMs. New corporate prospects are surfaced every morning in the team's Telegram group. On a **GO** command, an AI copywriting sub-workflow drafts a personalized HTML pitch; the team reviews it and either sends it (**OK**) or rewrites it (**EDIT**). A 6-day cadence sends up to 3 follow-ups automatically. Every inbound email is triaged: replies from known prospects update the CRM and alert the group, while emails from unknown senders are classified by a second AI sub-workflow, so only partnership-relevant messages become support tickets.

02 · The Triggers & On-Demand Testing

Module	Trigger Type & Production Cadence	On-Demand Testing Method
A · Prospect Ingestion	Schedule Trigger (A1), daily at 09:00, Europe/Paris timezone	Manual Trigger (A1bis) for one-click testing on the canvas.
B · Commands & Validation	Telegram Trigger (B1), webhook received through the Cloudflare tunnel at n8n.essec-liste-bde.com	Type **GO #1**, **EDIT #1 <text>** or **OK #1** in the Telegram group.
C · Follow-up Engine	Schedule Trigger (C1), every 6 days; SQL also enforces the 6-day gap per contact	Back-date *date_derniere_relance* by 7 days in SQL, then run C1 from the canvas.
D · Inbox Triage	Gmail Trigger (D1), polls unread inbox emails every minute (no webhook needed)	Reply to a sent pitch, or email the inbox from an unknown address (one partnership email, one advertising email).

Infrastructure note. The stack is self-hosted with **Docker Compose** (n8n 2.25.7, PostgreSQL 15.18, pgAdmin 9.16): pinned image versions, named volumes, a *pg_isready* healthcheck gating n8n's start-up, secrets and N8N_ENCRYPTION_KEY injected from a git-ignored .env file, ports bound to localhost only, and an init.sql that recreates the schema on a fresh clone. Telegram requires a public HTTPS webhook, provided by a named **Cloudflare Tunnel** (cloudflared installed as a Windows service, so it restarts automatically with the machine) that maps **n8n.essec-liste-bde.com** to localhost:5678. Outbound and inbound email use the Google Workspace mailbox **relation-liste-bde@essec-liste-bde.com**; the domain is authenticated with MX, SPF, DKIM and DMARC records so that pitches reach the inbox rather than spam.

03 · Architecture Overview

The solution is split into four n8n workflows. The main workflow holds the four operational modules (A to D); the two AI capabilities are isolated as reusable sub-workflows; a dedicated error workflow catches any unhandled failure. The full node-level diagram follows the data-flow table.

Workflow	Role	Key nodes
GOOD_v6_autom_bde (main)	Modules A, B, C, D: ingestion, Telegram commands, follow-ups, inbox triage	4 triggers · 4-way Switch router · 4 IF gates · Execute Workflow calls
Sub-Groq-Copywriter	AI service: writes the personalized HTML pitch from a contact record	Execute Workflow Trigger → Code (prompt) → HTTP Groq GPT-OSS 120B → Code (clean HTML)

Workflow	Role	Key nodes
Sub-AI-Mail-Classifier	AI service: decides whether an unknown sender's email is BDE / partnership related	Execute Workflow Trigger → Basic LLM Chain (written system prompt) + Groq Chat Model GPT-OSS 20B + Structured Output Parser → Code verdict | fallback
Error Handler - BDE PartnerBot	Global safety net set as the main workflow's Error Workflow	Error Trigger → PG insert error_log → Telegram admin alert

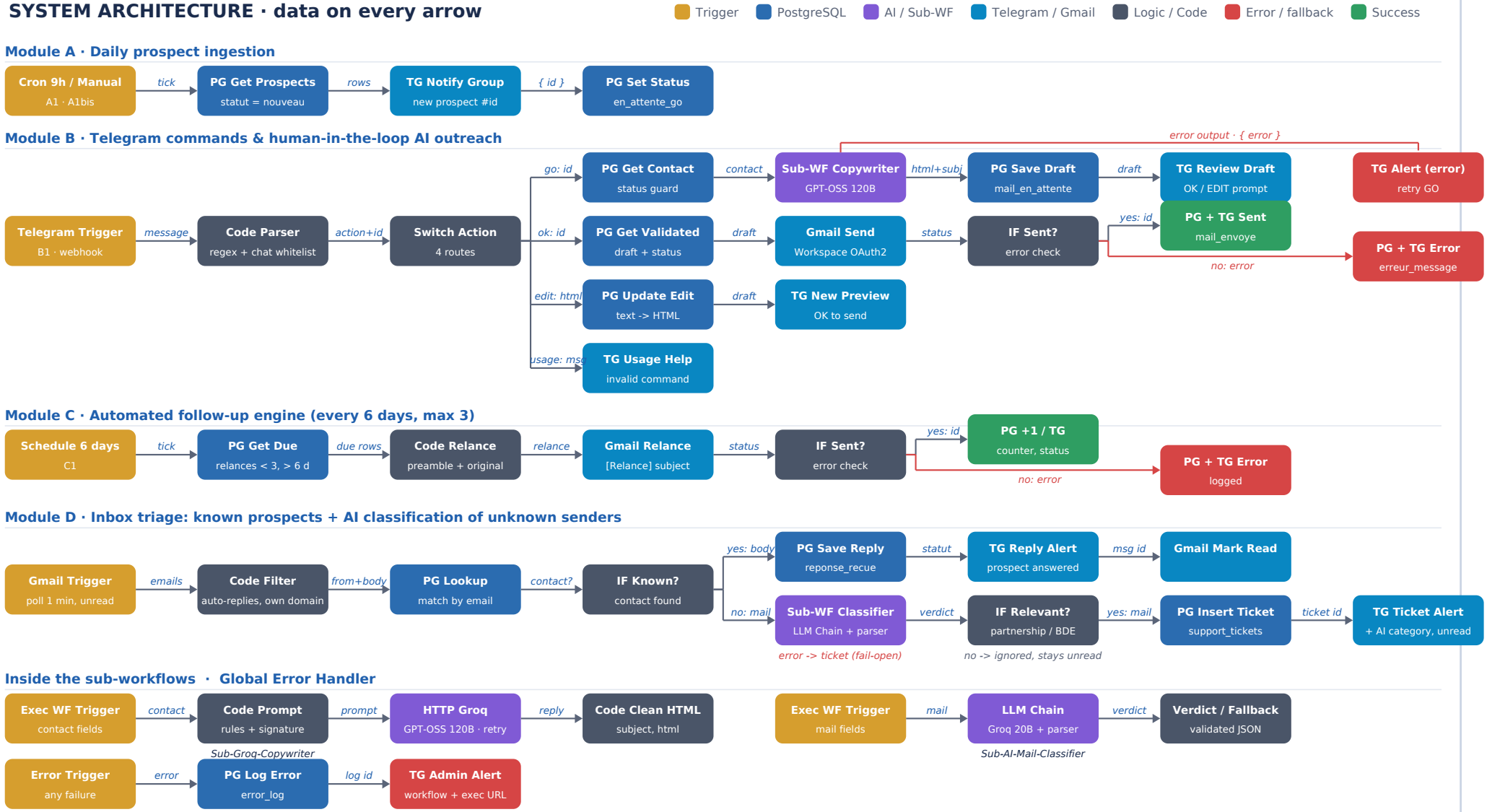
Contact lifecycle (PostgreSQL state machine): nouveau → en_attente_go → mail_en_attente_groupe → mail_envoye → relance_envoyee (up to 3) → pas_de_reponse, or reponse_recue as soon as the prospect answers. Every command checks the current status before acting, which prevents double sends.

04 · Data Flow Specifications per Module

Module	Component Sequence (Nodes)	Payload Traveling Between Nodes
A · Ingestion	Cron / Manual → PG Get Prospects → TG Notify → PG Update Status	{ id, email, nom, entreprise, produit_interesse }
B · GO	TG Trigger → Code Parser → Switch → PG Get Contact (status guard) → Sub-WF Copywriter → PG Save Draft → TG Review (error output → TG Alert)	{ action, contact_id } → { id, email, nom, entreprise, produit_interesse } → { mail_genere, mail_sujet }
B · OK / EDIT	PG Get Validated → Gmail Send → IF Sent → PG + TG (success or error) | PG Update Edit → TG New Preview	{ id, email, mail_genere, mail_sujet, error? } | { contact_id, new_html }
C · Follow-ups	Schedule (6 d) → PG Get Due (relances < 3) → Code Build Relance → Gmail Send → IF Sent → PG Counter +1 / Error log → TG	{ id, nombre_relances, mail_relance, sujet_relance }
D · Triage	Gmail Trigger → Code Filter → PG Lookup → IF Known → (yes) PG Save Reply → TG → Mark Read | (no) Sub-WF Classifier → IF Relevant → PG Insert Ticket → TG → Mark Unread	{ prospect_email, subject, body, thread_id, message_id } → { pertinent, categorie, raison } → { ticket_id }

Sub-workflow contracts. Sub-Groq-Copywriter receives { id, email, nom, entreprise, produit_interesse } and returns the same record plus { mail_genere, mail_sujet }. Sub-AI-Mail-Classifier receives { from_header, prospect_email, subject, body } and returns { pertinent, categorie, raison }; if the model output cannot be parsed, it defaults to *pertinent = true* so that no potential partner is ever silently dropped.

SYSTEM ARCHITECTURE · data on every arrow



05 · Level Aim & Cut Line

Target: Level 3 (Advanced). The workflow implements stateful CRM management across three relational tables, multi-step human-in-the-loop decision logic, two AI services modularized as sub-workflows (generation and classification), and resilient error triage with fallback branches and a global error workflow.

Cut line. If live demonstration time is constrained, features are cut in this order:

1. **AI classification of unknown senders**: fall back to the rule-based filter (only replies are ticketed).
2. **Inbound auto-ticketing**: keep reply tracking for known prospects, skip ticket logging.
3. **Automated follow-up engine (Module C)**: demonstrate the initial outreach only.
4. **EDIT command**: demonstrate GO and OK only.

Never cut: the core spine (PostgreSQL prospect load → Telegram notification → AI HTML draft via Sub-Groq-Copywriter → human Telegram confirmation → Gmail dispatch).

06 · Minimum Technical Requirements Mapping

Required Element	Node / Implementation in Workflow	Architectural Role & Resilience
If / Set node	B-OK-c IF Envoi Réussi · C5 IF Relance Envoyée · D4 IF Contact Trouvé · D4c IF Mail Pertinent	Checks Gmail delivery status, whether a sender exists in the CRM, and the AI relevance verdict.
Code node	B2 Parser · C3 Préparer Relance · D2 Filtrer Réponses · S1/S3 (copywriter) · K1/K3 (classifier)	Regex command parsing, plain-text to HTML conversion, auto-reply and own-domain filtering, prompt building, JSON parsing.
Router / Switch	B3 Switch Action (4 outputs, all reachable)	Routes chat intents to generation (go), dispatch (ok), revision (edit) or a usage/help reply for malformed commands (usage).
API / MCP connection	HTTP Request to Groq (OpenAI-compatible API) · Gmail OAuth2 · Telegram Bot API	REST integration with Groq Cloud inference, Google Workspace mail and the Telegram group.
Message trigger	B1 Telegram Trigger Commandes	Listens to the BDE Telegram supergroup through the Cloudflare tunnel; only the whitelisted chat ID is accepted.
Database persistence	PostgreSQL (Postgres-BDE): contacts, support_tickets, error_log	Stores prospects, drafts, lifecycle states, replies, follow-up counters, tickets and error history.
AI node (LLM)	K1 Basic LLM Chain + K1a Groq Chat Model GPT-OSS 20B (classification, written system prompt) · S2 Groq GPT-OSS 120B (copywriting)	Structured triage of inbound email; contextual B2B copywriting with enforced HTML format and signature.
AI output validation	K1b Structured Output Parser (JSON schema: pertinent, categorie, raison) · K2-Fallback branch	The verdict must match the schema; any failure or invalid output takes the fallback path and creates a ticket (fail-open).
Sub-workflow	Sub-Groq-Copywriter (B-GO-b) · Sub-AI-Mail-Classifier (D4b)	Both AI services are isolated, independently testable with pinned data, and reusable by other workflows.
Error handling & retries	Retry On Fail (3 tries, 1 s wait) on every Postgres, Gmail, Telegram and HTTP node · Error Handler workflow · error outputs on Execute Workflow and LLM Chain nodes	Unhandled failures are logged in error_log and pushed to Telegram; AI failures fall back to a safe path (GO alert, fail-open ticket).

07 · Robustness & Security Choices

Risk	Mitigation implemented
Unauthorized bot commands	Code parser only accepts messages from the BDE group's chat ID; all other chats are ignored.
Invalid input	Malformed commands (GO 1, OK #abc, EDIT #3 without text, /help) receive a usage message listing the valid commands; normal group chat is ignored.
Double sending / acting on stale state	Every GO, OK and EDIT query is filtered on the contact's current status; a sent pitch cannot be regenerated or re-sent.
SQL injection and broken parameters	All queries use positional parameters passed as arrays, so commas and quotes in emails never break or alter a query.
Telegram formatting failures	Dynamic values are escaped before sending, so underscores in statuses or email addresses cannot break a notification.
Deliverability	Domain authenticated with SPF, DKIM and DMARC; sender name and signature enforced in every email.